JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Thread

java.lang.Object
    java.lang.Thread

All Implemented Interfaces:

`Runnable`

Direct Known Subclasses:

`ForkJoinWorkerThread`

```
public class Thread
extends Object
implements Runnable
```

A *thread* is a thread of execution in a program. The Java virtual machine allows an application to have multiple threads of execution running concurrently.

`Thread` defines constructors and a `Thread.Builder` to create threads. Starting a thread schedules it to execute its `run` method. The newly started thread executes concurrently with the thread that caused it to start.

A thread *terminates* if either its `run` method completes normally, or if its `run` method completes abruptly and the appropriate uncaught exception handler completes normally or abruptly. With no code left to run, the thread has completed execution. The `isAlive` method can be used to test if a started thread has terminated. The `join` method can be used to wait for a thread to terminate.

Threads have a unique identifier and a name. The identifier is generated when a `Thread` is created and cannot be changed. The thread name can be specified when creating a thread or can be changed at a later time.

Threads support `ThreadLocal` variables. These are variables that are local to a thread, meaning a thread can have a copy of a variable that is set to a value that is independent of the value set by other threads. `Thread` also supports `InheritableThreadLocal` variables that are thread local variables that are inherited at thread creation time from the parent `Thread`. `Thread` supports a special inheritable thread local for the thread context-class-loader.

Platform Threads

`Thread` supports the creation of *platform threads* that are typically mapped 1:1 to kernel threads scheduled by the operating system. Platform threads will usually have a large stack and other resources that are maintained by the operating system. Platforms threads are suitable for executing all types of tasks but may be a limited resource.

Platform threads get an automatically generated thread name by default.

Platform threads are designated *daemon* or *non-daemon* threads. When the Java virtual machine starts up, there is usually one non-daemon thread (the thread that typically calls the application's `main` method). The shutdown sequence begins when all started non-daemon threads have terminated. Unstarted non-daemon threads do not prevent the shutdown sequence from beginning.

In addition to the daemon status, platform threads have a thread priority and are members of a thread group.

Virtual Threads

`Thread` also supports the creation of *virtual threads*. Virtual threads are typically *user-mode threads* scheduled by the Java runtime rather than the operating system. Virtual threads will typically require few resources and a single Java virtual machine may support millions of virtual threads. Virtual threads are suitable for executing tasks that spend most of the time blocked, often waiting for I/O operations to complete. Virtual threads are not intended for long running CPU intensive operations.

Virtual threads typically employ a small set of platform threads used as *carrier threads*. Locking and I/O operations are examples of operations where a carrier thread may be re-scheduled from one virtual thread to another. Code executing in a virtual thread is not aware of the underlying carrier thread. The currentThread() method, used to obtain a reference to the *current thread*, will always return the `Thread` object for the virtual thread.

Virtual threads do not have a thread name by default. The `getName` method returns the empty string if a thread name is not set.

Virtual threads are daemon threads and so do not prevent the shutdown sequence from beginning. Virtual threads have a fixed thread priority that cannot be changed.

Creating And Starting Threads

`Thread` defines public constructors for creating platform threads and the `start` method to schedule threads to execute. `Thread` may be extended for customization and other advanced reasons although most applications should have little need to do this.

`Thread` defines a `Thread.Builder` API for creating and starting both platform and virtual threads. The following are examples that use the builder:

```
Runnable runnable = ...

// Start a daemon thread to run a task
Thread thread = Thread.ofPlatform().daemon().start(runnable);

// Create an unstarted thread with name "duke", its start() method
// must be invoked to schedule it to execute.
Thread thread = Thread.ofPlatform().name("duke").unstarted(runnable);

// A ThreadFactory that creates daemon threads named "worker-0", "worker-1", ..
ThreadFactory factory = Thread.ofPlatform().daemon().name("worker-", 0).factory

// Start a virtual thread to run a task
Thread thread = Thread.ofVirtual().start(runnable);

// A ThreadFactory that creates virtual threads
ThreadFactory factory = Thread.ofVirtual().factory();
```

Inheritance When Creating Threads

A `Thread` created with one of the public constructors inherits the daemon status and thread priority from the parent thread at the time that the child `Thread` is created. The thread group is also inherited when not provided to the constructor. When using a `Thread.Builder` to create a platform thread, the daemon status, thread priority, and thread group are inherited when not set on the builder. As with the constructors, inheriting from the parent thread is done when the child `Thread` is created.

A `Thread` inherits its initial values of inheritable-thread-local variables (including the context class loader) from the parent thread values at the time that the child `Thread` is created. The 5-param constructor can be used to create a thread that does not inherit its initial values from the constructing thread. When using a `Thread.Builder`, the `inheritInheritableThreadLocals` method can be used to select if the initial values are inherited.

Thread Interruption

A `Thread` has an *interrupted status* which serves as a "request" for code executing in the thread to "stop or cancel its current activity". The interrupted status is set by invoking the target thread's `interrupt()` method. Many methods that cause a thread to block or wait are *interruptible*, meaning they detect that the thread's interrupted status is set and cause execution to return early from the method, usually by throwing an exception.

If a thread executing `Thread.sleep` or `Object.wait` is interrupted then it causes the method to throw `InterruptedException`. Methods that throw `InterruptedException` do so after first clearing the interrupted status. Code that catches `InterruptedException` should rethrow the exception, or restore the current thread's interrupted status, with `Thread.currentThread().interrupt()`, before continuing normally or handling it by throwing another type of exception. Code that throws another type of exception with the `InterruptedException` as cause, or the `InterruptedException` as a suppressed exception, should also restore the interrupted status before throwing the exception.

If a thread executing a blocking I/O operation on an `InterruptibleChannel` is interrupted then it causes the channel to be closed, and the blocking I/O operation to throw `ClosedByInterruptException` with the thread's interrupted status set. If a thread blocked in a selection operation is interrupted then it causes the selection operation to return early, with the thread's interrupted status set.

Code that doesn't invoke any interruptible methods can still respond to interrupt by polling the current thread's interrupted status with `Thread.currentThread().isInterrupted()`.

In addition to the `interrupt()` and `isInterrupted()` methods, `Thread` also defines the static `Thread.interrupted()` method to test the current thread's interrupted status and clear it. It should be rare to need to use this method.

Null Handling

Unless otherwise specified, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

Implementation Note:

In the JDK Reference Implementation, the virtual thread scheduler may be configured with the following system properties:

System property	Description
`jdk.virtualThreadScheduler.parallelism`	The scheduler's target parallelism. This is the number of platform threads available for scheduling virtual threads. It defaults to the number of available processors.
`jdk.virtualThreadScheduler.maxPoolSize`	The maximum number of platform threads available to

System property	Description
	the scheduler. It defaults to 256.

The virtual thread scheduler can be monitored and managed with the `jdk.management.VirtualThreadSchedulerMXBean` management interface.

Since:

1.0

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
`static interface`	`Thread.Builder`	A builder for `Thread` and `ThreadFactory` objects.
`static enum`	`Thread.State`	A thread state.
`static interface`	`Thread.UncaughtExceptionHandler`	Interface for handlers invoked when a `Thread` abruptly terminates due to an uncaught exception.

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	`MAX_PRIORITY`	The maximum priority that a thread can have.
`static final int`	`MIN_PRIORITY`	The minimum priority that a thread can have.
`static final int`	`NORM_PRIORITY`	The default priority that is assigned to a thread.

Constructor Summary

Constructors

Constructor	Description

Thread()	Initializes a new platform Thread.
Thread(Runnable task)	Initializes a new platform Thread.
Thread(Runnable task, String name)	Initializes a new platform Thread.
Thread(String name)	Initializes a new platform Thread.
Thread(ThreadGroup group, Runnable task)	Initializes a new platform Thread.
Thread(ThreadGroup group, Runnable task, String name)	Initializes a new platform Thread so that it has task as its run object, has the specified name as its name, and belongs to the thread group referred to by group.
Thread(ThreadGroup group, Runnable task, String name, long stackSize)	Initializes a new platform Thread so that it has task as its run object, has the specified name as its name, and belongs to the thread group referred to by group, and has the specified *stack size*.
Thread(ThreadGroup group, Runnable task, String name, long stackSize, boolean inheritInheritableThreadLo	Initializes a new platform Thread so that it has task as its run object, has the specified name as its name, belongs to the thread group referred to by group, has the specified stackSize, and inherits initial values for inheritable thread-local variables if inheritThreadLocals is true.
Thread(ThreadGroup group, String name)	Initializes a new platform Thread.

Method Summary

All Methods	Static Methods	

Modifier and Type	Method	Description
static int	**activeCount**()	Returns an estimate of the number of live

		platform threads in the current thread's thread group and its subgroups.
final void	**checkAccess**()	**Deprecated, for removal: This API element is subject to removal in a future version.** This method originally determined if the currently running thread had permission to modify this thread.
protected Object	**clone**()	Throws CloneNotSupportedException as a Thread can not be meaningfully cloned.
static Thread	**currentThread**()	Returns the Thread object for the current thread.
static void	**dumpStack**()	Prints a stack trace of the current thread to the standard error stream.
static int	**enumerate**(Thread[] tarray)	Copies into the specified array every live platform thread in the current thread's thread group and its subgroups.
static Map<Thread, StackTraceElement[]>	**getAllStackTraces**()	Returns a map of stack traces for all live platform threads.
ClassLoader	**getContextClassLoader**()	Returns the context ClassLoader for this thread.
static Thread.UncaughtExcep	**getDefaultUncaughtExcept**	Returns the default handler invoked when a

		thread abruptly terminates due to an uncaught exception.
long	**getId**()	**Deprecated.** This method is not final and may be overridden to return a value that is not the thread ID.
final String	**getName**()	Returns this thread's name.
final int	**getPriority**()	Returns this thread's priority.
StackTraceElement[]	**getStackTrace**()	Returns an array of stack trace elements representing the stack dump of this thread.
Thread.State	**getState**()	Returns the state of this thread.
final ThreadGroup	**getThreadGroup**()	Returns the thread's thread group or `null` if the thread has terminated.
Thread.UncaughtExcep	**getUncaughtExceptionHand**	Returns the handler invoked when this thread abruptly terminates due to an uncaught exception.
static boolean	**holdsLock**(Object obj)	Returns `true` if and only if the current thread holds the monitor lock on the specified object.
void	**interrupt**()	Interrupts this thread.
static boolean	**interrupted**()	Tests whether the current thread has been interrupted.

final boolean	**isAlive**()	Tests if this thread is alive.
final boolean	**isDaemon**()	Tests if this thread is a daemon thread.
boolean	**isInterrupted**()	Tests whether this thread has been interrupted.
final boolean	**isVirtual**()	Returns `true` if this thread is a virtual thread.
final void	**join**()	Waits for this thread to terminate.
final void	**join**(long millis)	Waits at most `millis` milliseconds for this thread to terminate.
final void	**join**(long millis, int nanos)	Waits at most `millis` milliseconds plus `nanos` nanoseconds for this thread to terminate.
final boolean	**join**(Duration duration)	Waits for this thread to terminate for up to the given waiting duration.
static Thread.Builder.OfPla	**ofPlatform**()	Returns a builder for creating a platform `Thread` or `ThreadFactory` that creates platform threads.
static Thread.Builder.OfVir	**ofVirtual**()	Returns a builder for creating a virtual `Thread` or `ThreadFactory` that creates virtual threads.
static void	**onSpinWait**()	Indicates that the caller is momentarily unable to

		progress, until the occurrence of one or more actions on the part of other activities.
void	**run**()	This method is run by the thread when it executes.
void	**setContextClassLoader**(ClassLoader cl)	Sets the context ClassLoader for this thread.
final void	**setDaemon**(boolean on)	Marks this thread as either a *daemon* or *non-daemon* thread.
static void	**setDefaultUncaughtExcept**(Thread.UncaughtExceptio	Set the default handler invoked when a thread abruptly terminates due to an uncaught exception, and no other handler has been defined for that thread.
final void	**setName**(String name)	Changes the name of this thread to be equal to the argument name.
final void	**setPriority**(int newPriority)	Changes the priority of this thread.
void	**setUncaughtExceptionHand**(Thread.UncaughtExceptio	Set the handler invoked when this thread abruptly terminates due to an uncaught exception.
static void	**sleep**(long millis)	Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds, subject to

		the precision and accuracy of system timers and schedulers.
static void	**sleep**(long millis, int nanos)	Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds plus the specified number of nanoseconds, subject to the precision and accuracy of system timers and schedulers.
static void	**sleep**(Duration duration)	Causes the currently executing thread to sleep (temporarily cease execution) for the specified duration, subject to the precision and accuracy of system timers and schedulers.
void	**start**()	Schedules this thread to begin execution.
static Thread	**startVirtualThread**(Runnable task)	Creates a virtual thread to execute a task and schedules it to execute.
final long	**threadId**()	Returns the identifier of this Thread.
String	**toString**()	Returns a string representation of this thread.
static void	**yield**()	

	A hint to the scheduler that the current thread is willing to yield its current use of a processor.

Methods declared in class Object

`equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Field Details

MIN_PRIORITY

```
public static final int MIN_PRIORITY
```

The minimum priority that a thread can have.

See Also:

Constant Field Values

NORM_PRIORITY

```
public static final int NORM_PRIORITY
```

The default priority that is assigned to a thread.

See Also:

Constant Field Values

MAX_PRIORITY

```
public static final int MAX_PRIORITY
```

The maximum priority that a thread can have.

See Also:

Constant Field Values

Constructor Details

Thread

```
public Thread()
```

Initializes a new platform `Thread`. This constructor has the same effect as `Thread(null, null, gname)`, where `gname` is a newly generated name. Automatically generated names are of the form `"Thread-"+n`, where *n* is an integer.

This constructor is only useful when extending `Thread` to override the `run()` method.

See Also:

Inheritance when creating threads

Thread

```
public Thread(Runnable task)
```

Initializes a new platform `Thread`. This constructor has the same effect as `Thread(null, task, gname)`, where `gname` is a newly generated name. Automatically generated names are of the form `"Thread-"+n`, where *n* is an integer.

For a non-null task, invoking this constructor directly is equivalent to:

```
Thread.ofPlatform().unstarted(task);
```

Parameters:

`task` - the object whose `run` method is invoked when this thread is started. If `null`, this classes `run` method does nothing.

See Also:

Inheritance when creating threads

Thread

```
public Thread(ThreadGroup group,
              Runnable task)
```

Initializes a new platform `Thread`. This constructor has the same effect as `Thread(group, task, gname)`, where `gname` is a newly generated name. Automatically generated names are of the form `"Thread-"+n`, where *n* is an integer.

For a non-null group and task, invoking this constructor directly is equivalent to:

```
Thread.ofPlatform().group(group).unstarted(task);
```

Parameters:

`group` - the thread group. If `null` the group is set to the current thread's thread group.

`task` - the object whose `run` method is invoked when this thread is started. If `null`, this thread's run method is invoked.

See Also:

Inheritance when creating threads

Thread

```
public Thread(String name)
```

Initializes a new platform `Thread`. This constructor has the same effect as `Thread (null, null, name)`.

This constructor is only useful when extending `Thread` to override the `run()` method.

Parameters:

`name` - the name of the new thread

See Also:

Inheritance when creating threads

Thread

```
public Thread(ThreadGroup group,
              String name)
```

Initializes a new platform `Thread`. This constructor has the same effect as `Thread (group, null, name)`.

This constructor is only useful when extending `Thread` to override the `run()` method.

Parameters:

`group` - the thread group. If `null`, the group is set to the current thread's thread group.

`name` - the name of the new thread

See Also:

Inheritance when creating threads

Thread

```
public Thread(Runnable task,
              String name)
```

Initializes a new platform `Thread`. This constructor has the same effect as `Thread(null, task, name)`.

For a non-null task and name, invoking this constructor directly is equivalent to:

```
Thread.ofPlatform().name(name).unstarted(task);
```

Parameters:

`task` - the object whose `run` method is invoked when this thread is started. If `null`, this thread's run method is invoked.

`name` - the name of the new thread

See Also:

Inheritance when creating threads

Thread

```
public Thread(ThreadGroup group,
              Runnable task,
              String name)
```

Initializes a new platform `Thread` so that it has `task` as its run object, has the specified `name` as its name, and belongs to the thread group referred to by `group`.

The priority of the newly created thread is the smaller of priority of the thread creating it and the maximum permitted priority of the thread group. The method setPriority may be used to change the priority to a new value.

The newly created thread is initially marked as being a daemon thread if and only if the thread creating it is currently marked as a daemon thread. The method setDaemon may be used to change whether or not a thread is a daemon.

For a non-null group, task, and name, invoking this constructor directly is equivalent to:

```
Thread.ofPlatform().group(group).name(name).unstarted(task);
```

Parameters:

`group` - the thread group. If `null`, the group is set to the current thread's thread group.

`task` - the object whose `run` method is invoked when this thread is started. If `null`, this thread's run method is invoked.

`name` - the name of the new thread

See Also:

Inheritance when creating threads

Thread

```
public Thread(ThreadGroup group,
              Runnable task,
              String name,
              long stackSize)
```

Initializes a new platform `Thread` so that it has `task` as its run object, has the specified `name` as its name, and belongs to the thread group referred to by `group`, and has the specified *stack size*.

This constructor is identical to `Thread(ThreadGroup,Runnable,String)` with the exception of the fact that it allows the thread stack size to be specified. The stack size is the approximate number of bytes of address space that the virtual machine is to allocate for this thread's stack. **The effect of the `stackSize` parameter, if any, is highly platform dependent.**

On some platforms, specifying a higher value for the `stackSize` parameter may allow a thread to achieve greater recursion depth before throwing a `StackOverflowError`. Similarly, specifying a lower value may allow a greater number of threads to exist concurrently without throwing an `OutOfMemoryError` (or other internal error). The details of the relationship between the value of the `stackSize` parameter and the maximum recursion depth and concurrency level are platform-dependent. **On some platforms, the value of the `stackSize` parameter may have no effect whatsoever.**

The virtual machine is free to treat the `stackSize` parameter as a suggestion. If the specified value is unreasonably low for the platform, the virtual machine may instead use some platform-specific minimum value; if the specified value is unreasonably high, the virtual machine may instead use some platform-specific maximum. Likewise, the virtual machine is free to round the specified value up or down as it sees fit (or to ignore it completely).

Specifying a value of zero for the `stackSize` parameter will cause this constructor to behave exactly like the `Thread(ThreadGroup, Runnable, String)` constructor.

Due to the platform-dependent nature of the behavior of this constructor, extreme care should be exercised in its use. The thread stack size necessary to perform a given computation will likely vary from one JRE implementation to another. In light of this variation, careful tuning of the stack size parameter may be required, and the tuning may need to be repeated for each JRE implementation on which an application is to run.

Implementation note: Java platform implementers are encouraged to document their implementation's behavior with respect to the `stackSize` parameter.

For a non-null group, task, and name, invoking this constructor directly is equivalent to:

```
Thread.ofPlatform().group(group).name(name).stackSize(stackSize).unstarted(task)
```

Parameters:

`group` - the thread group. If `null`, the group is set to the current thread's thread group.

`task` - the object whose `run` method is invoked when this thread is started. If `null`, this thread's run method is invoked.

`name` - the name of the new thread

`stackSize` - the desired stack size for the new thread, or zero to indicate that this parameter is to be ignored.

Since:

1.4

See Also:

Inheritance when creating threads

Thread

```
public Thread(ThreadGroup group,
              Runnable task,
              String name,
              long stackSize,
              boolean inheritInheritableThreadLocals)
```

Initializes a new platform `Thread` so that it has `task` as its run object, has the specified `name` as its name, belongs to the thread group referred to by `group`, has the specified `stackSize`, and inherits initial values for inheritable thread-local variables if `inheritThreadLocals` is `true`.

This constructor is identical to `Thread(ThreadGroup,Runnable,String,long)` with the added ability to suppress, or not, the inheriting of initial values for inheritable thread-local variables from the constructing thread. This allows for finer grain control over inheritable thread-locals. Care must be taken when passing a value of `false` for `inheritThreadLocals`, as it may lead to unexpected behavior if the new thread executes code that expects a specific thread-local value to be inherited.

Specifying a value of `true` for the `inheritThreadLocals` parameter will cause this constructor to behave exactly like the `Thread(ThreadGroup, Runnable, String, long)` constructor.

For a non-null group, task, and name, invoking this constructor directly is equivalent to:

```
Thread.ofPlatform()
      .group(group)
      .name(name)
      .stackSize(stackSize)
      .inheritInheritableThreadLocals(inheritInheritableThreadLocals)
      .unstarted(task);
```

Parameters:

`group` - the thread group. If `null`, the group is set to the current thread's thread group.

`task` - the object whose `run` method is invoked when this thread is started. If `null`, this thread's run method is invoked.

`name` - the name of the new thread

`stackSize` - the desired stack size for the new thread, or zero to indicate that this parameter is to be ignored

`inheritInheritableThreadLocals` - if `true`, inherit initial values for inheritable thread-locals from the constructing thread, otherwise no initial values are inherited

Since:

9

See Also:

Inheritance when creating threads

Method Details

currentThread

`public static Thread currentThread()`

Returns the Thread object for the current thread.

Returns:

the current thread

yield

```
public static void yield()
```

A hint to the scheduler that the current thread is willing to yield its current use of a processor. The scheduler is free to ignore this hint.

Yield is a heuristic attempt to improve relative progression between threads that would otherwise over-utilise a CPU. Its use should be combined with detailed profiling and benchmarking to ensure that it actually has the desired effect.

It is rarely appropriate to use this method. It may be useful for debugging or testing purposes, where it may help to reproduce bugs due to race conditions. It may also be useful when designing concurrency control constructs such as the ones in the `java.util.concurrent.locks` package.

sleep

```
public static void sleep(long millis)
                  throws InterruptedException
```

Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds, subject to the precision and accuracy of system timers and schedulers. The thread does not lose ownership of any monitors.

Parameters:

`millis` - the length of time to sleep in milliseconds

Throws:

`IllegalArgumentException` - if the value of `millis` is negative

`InterruptedException` - if any thread has interrupted the current thread. The *interrupted status* of the current thread is cleared when this exception is thrown.

sleep

```
public static void sleep(long millis,
                         int nanos)
                  throws InterruptedException
```

Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds plus the specified number of nanoseconds, subject to the precision and accuracy of system timers and schedulers. The thread does not lose ownership of any monitors.

Parameters:

`millis` - the length of time to sleep in milliseconds

`nanos` - `0-999999` additional nanoseconds to sleep

Throws:

`IllegalArgumentException` - if the value of `millis` is negative, or the value of `nanos` is not in the range `0-999999`

`InterruptedException` - if any thread has interrupted the current thread. The *interrupted status* of the current thread is cleared when this exception is thrown.

sleep

```
public static void sleep(Duration duration)
                  throws InterruptedException
```

Causes the currently executing thread to sleep (temporarily cease execution) for the specified duration, subject to the precision and accuracy of system timers and schedulers. This method is a no-op if the duration is negative.

Parameters:

`duration` - the duration to sleep

Throws:

`InterruptedException` - if the current thread is interrupted while sleeping. The *interrupted status* of the current thread is cleared when this exception is thrown.

Since:

19

onSpinWait

```
public static void onSpinWait()
```

Indicates that the caller is momentarily unable to progress, until the occurrence of one or more actions on the part of other activities. By invoking this method within each iteration of a spin-wait loop construct, the calling thread indicates to the runtime that it is busy-waiting. The runtime may take action to improve the performance of invoking spin-wait loop constructions.

API Note:

As an example consider a method in a class that spins in a loop until some flag is set outside of that method. A call to the `onSpinWait` method should be placed inside the spin loop.

```
    class EventHandler {
        volatile boolean eventNotificationNotReceived;
```

```
        void waitForEventAndHandleIt() {
            while ( eventNotificationNotReceived ) {
                Thread.onSpinWait();
            }
            readAndProcessEvent();
        }

        void readAndProcessEvent() {
            // Read event from some source and process it
             . . .
        }
    }
```

The code above would remain correct even if the `onSpinWait` method was not called at all. However on some architectures the Java Virtual Machine may issue the processor instructions to address such code patterns in a more beneficial way.

Since:

9

ofPlatform

`public static Thread.Builder.OfPlatform ofPlatform()`

Returns a builder for creating a platform `Thread` or `ThreadFactory` that creates platform threads.

API Note:

The following are examples using the builder:

```
// Start a daemon thread to run a task
Thread thread = Thread.ofPlatform().daemon().start(runnable);

// Create an unstarted thread with name "duke", its start() method
// must be invoked to schedule it to execute.
Thread thread = Thread.ofPlatform().name("duke").unstarted(runnable);

// A ThreadFactory that creates daemon threads named "worker-0", "worker-1",
ThreadFactory factory = Thread.ofPlatform().daemon().name("worker-", 0).fact
```

Returns:

A builder for creating `Thread` or `ThreadFactory` objects.

Since:

21

ofVirtual

```
public static Thread.Builder.OfVirtual ofVirtual()
```

Returns a builder for creating a virtual `Thread` or `ThreadFactory` that creates virtual threads.

API Note:

The following are examples using the builder:

```
// Start a virtual thread to run a task.
Thread thread = Thread.ofVirtual().start(runnable);

// A ThreadFactory that creates virtual threads
ThreadFactory factory = Thread.ofVirtual().factory();
```

Returns:

A builder for creating `Thread` or `ThreadFactory` objects.

Since:

21

clone

```
protected Object clone()
                throws CloneNotSupportedException
```

Throws CloneNotSupportedException as a Thread can not be meaningfully cloned. Construct a new Thread instead.

Overrides:

`clone` in class `Object`

Returns:

a clone of this instance.

Throws:

`CloneNotSupportedException` - always

See Also:

`Cloneable`

startVirtualThread

```
public static Thread startVirtualThread(Runnable task)
```

Creates a virtual thread to execute a task and schedules it to execute.

This method is equivalent to:

```
Thread.ofVirtual().start(task);
```

Parameters:

`task` - the object to run when the thread executes

Returns:

a new, and started, virtual thread

Since:

21

See Also:

Inheritance when creating threads

isVirtual

```
public final boolean isVirtual()
```

Returns `true` if this thread is a virtual thread. A virtual thread is scheduled by the Java virtual machine rather than the operating system.

Returns:

`true` if this thread is a virtual thread

Since:

21

start

```
public void start()
```

Schedules this thread to begin execution. The thread will execute independently of the current thread.

A thread can be started at most once. In particular, a thread can not be restarted after it has terminated.

Throws:

`IllegalThreadStateException` - if the thread was already started

run

```
public void run()
```

This method is run by the thread when it executes. Subclasses of `Thread` may override this method.

This method is not intended to be invoked directly. If this thread is a platform thread created with a `Runnable` task then invoking this method will invoke the task's `run` method. If this thread is a virtual thread then invoking this method directly does nothing.

Specified by:

`run` in interface `Runnable`

Implementation Requirements:

The default implementation executes the `Runnable` task that the `Thread` was created with. If the thread was created without a task then this method does nothing.

interrupt

```
public void interrupt()
```

Interrupts this thread.

If this thread is blocked in an invocation of the `wait()`, `wait(long)`, or `wait(long, int)` methods of the `Object` class, or of the `join()`, `join(long)`, `join(long, int)`, `sleep(long)`, or `sleep(long, int)` methods of this class, then its interrupted status will be cleared and it will receive an `InterruptedException`.

If this thread is blocked in an I/O operation upon an `InterruptibleChannel` then the channel will be closed, the thread's interrupted status will be set, and the thread will receive a `ClosedByInterruptException`.

If this thread is blocked in a `Selector` then the thread's interrupted status will be set and it will return immediately from the selection operation, possibly with a non-zero value, just as if the selector's `wakeup` method were invoked.

If none of the previous conditions hold then this thread's interrupted status will be set.

Interrupting a thread that is not alive need not have any effect.

Implementation Note:

In the JDK Reference Implementation, interruption of a thread that is not alive still records that the interrupt request was made and will report it via `interrupted()` and `isInterrupted()`.

See Also:

Thread Interruption, `isInterrupted()`

interrupted

```
public static boolean interrupted()
```

Tests whether the current thread has been interrupted. The *interrupted status* of the thread is cleared by this method. In other words, if this method were to be called twice in succession, the second call would return false (unless the current thread were interrupted again, after the first call had cleared its interrupted status and before the second call had examined it).

API Note:

It should be rare to use this method directly. It is intended for cases that detect thread interruption and clear the interrupted status before throwing `InterruptedException`. It may also be useful for cases that implement an *uninterruptible* method that makes use of an *interruptible* method such as `LockSupport.park()`. The `interrupted()` method can be used to test if interrupted and clear the interrupted status to allow the code retry the *interruptible* method. The *uninterruptible* method should restore the interrupted status before it completes.

Returns:

`true` if the current thread has been interrupted; `false` otherwise.

See Also:

Thread Interruption, `isInterrupted()`

isInterrupted

```
public boolean isInterrupted()
```

Tests whether this thread has been interrupted. The *interrupted status* of the thread is unaffected by this method.

Returns:

`true` if this thread has been interrupted; `false` otherwise.

See Also:

Thread Interruption, `interrupt()`

isAlive

```
public final boolean isAlive()
```

Tests if this thread is alive. A thread is alive if it has been started and has not yet terminated.

Returns:

`true` if this thread is alive; `false` otherwise.

setPriority

```
public final void setPriority(int newPriority)
```

Changes the priority of this thread. For platform threads, the priority is set to the smaller of the specified `newPriority` and the maximum permitted priority of the thread's thread group. The priority of a virtual thread is always `NORM_PRIORITY` and `newPriority` is ignored.

Parameters:

`newPriority` - the new thread priority

Throws:

`IllegalArgumentException` - if the priority is not in the range `MIN_PRIORITY` to `MAX_PRIORITY`.

See Also:

`setPriority(int)`, `ThreadGroup.getMaxPriority()`

getPriority

```
public final int getPriority()
```

Returns this thread's priority.

The priority of a virtual thread is always `NORM_PRIORITY`.

Returns:

this thread's priority.

See Also:

`setPriority(int)`

setName

```
public final void setName(String name)
```

Changes the name of this thread to be equal to the argument `name`.

Implementation Note:

In the JDK Reference Implementation, if this thread is the current thread, and it's a platform thread that was not attached to the VM with the Java Native Interface AttachCurrentThread function, then this method will set the operating system thread name. This may be useful for debugging and troubleshooting purposes.

Parameters:

`name` - the new name for this thread.

External Specifications

Java Native Interface Specification

See Also:

`getName()`

getName

```
public final String getName()
```

Returns this thread's name.

Returns:

this thread's name.

See Also:

`setName(String)`

getThreadGroup

```
public final ThreadGroup getThreadGroup()
```

Returns the thread's thread group or `null` if the thread has terminated.

The thread group returned for a virtual thread is the special *ThreadGroup for virtual threads*.

Returns:

this thread's thread group or `null`

activeCount

```
public static int activeCount()
```

Returns an estimate of the number of live platform threads in the current thread's thread group and its subgroups. Virtual threads are not included in the estimate.

The value returned is only an estimate because the number of threads may change dynamically while this method traverses internal data structures, and might be affected by the presence of certain system threads. This method is intended primarily for debugging and monitoring purposes.

Returns:

an estimate of the number of live platform threads in the current thread's thread group and in any other thread group that has the current thread's thread group as an ancestor

enumerate

```
public static int enumerate(Thread[] tarray)
```

Copies into the specified array every live platform thread in the current thread's thread group and its subgroups. This method simply invokes the

`ThreadGroup.enumerate(Thread[])` method of the current thread's thread group. Virtual threads are not enumerated by this method.

An application might use the activeCount method to get an estimate of how big the array should be, however *if the array is too short to hold all the threads, the extra threads are silently ignored.* If it is critical to obtain every live thread in the current thread's thread group and its subgroups, the invoker should verify that the returned int value is strictly less than the length of `tarray`.

Due to the inherent race condition in this method, it is recommended that the method only be used for debugging and monitoring purposes.

Parameters:

`tarray` - an array into which to put the list of threads

Returns:

the number of threads put into the array

join

```
public final void join(long millis)
                throws InterruptedException
```

Waits at most `millis` milliseconds for this thread to terminate. A timeout of `0` means to wait forever. This method returns immediately, without waiting, if the thread has not been `started`.

Implementation Note:

This implementation uses a loop of `this.wait` calls conditioned on `this.isAlive`. As a thread terminates the `this.notifyAll` method is invoked. It is recommended that applications not use `wait`, `notify`, or `notifyAll` on `Thread` instances.

Parameters:

`millis` - the time to wait in milliseconds

Throws:

`IllegalArgumentException` - if the value of `millis` is negative

`InterruptedException` - if any thread has interrupted the current thread. The *interrupted status* of the current thread is cleared when this exception is thrown.

join

```
public final void join(long millis,
                       int nanos)
                throws InterruptedException
```

Waits at most `millis` milliseconds plus `nanos` nanoseconds for this thread to terminate. If both arguments are `0`, it means to wait forever. This method returns immediately, without waiting, if the thread has not been `started`.

Implementation Note:

This implementation uses a loop of `this.wait` calls conditioned on `this.isAlive`. As a thread terminates the `this.notifyAll` method is invoked. It is recommended that applications not use `wait`, `notify`, or `notifyAll` on `Thread` instances.

Parameters:

`millis` - the time to wait in milliseconds

`nanos` - `0-999999` additional nanoseconds to wait

Throws:

`IllegalArgumentException` - if the value of `millis` is negative, or the value of `nanos` is not in the range `0-999999`

`InterruptedException` - if any thread has interrupted the current thread. The *interrupted status* of the current thread is cleared when this exception is thrown.

join

```
public final void join()
                throws InterruptedException
```

Waits for this thread to terminate.

An invocation of this method behaves in exactly the same way as the invocation

> `join(0)`

Throws:

`InterruptedException` - if any thread has interrupted the current thread. The *interrupted status* of the current thread is cleared when this exception is thrown.

join

```
public final boolean join(Duration duration)
                  throws InterruptedException
```

Waits for this thread to terminate for up to the given waiting duration.

This method does not wait if the duration to wait is less than or equal to zero. In this case, the method just tests if the thread has terminated.

Parameters:

`duration` - the maximum duration to wait

Returns:

`true` if the thread has terminated, `false` if the thread has not terminated

Throws:

`InterruptedException` - if the current thread is interrupted while waiting. The *interrupted status* of the current thread is cleared when this exception is thrown.

`IllegalThreadStateException` - if this thread has not been started.

Since:

19

dumpStack

```
public static void dumpStack()
```

Prints a stack trace of the current thread to the standard error stream. This method is useful for debugging.

setDaemon

```
public final void setDaemon(boolean on)
```

Marks this thread as either a *daemon* or *non-daemon* thread. The shutdown sequence begins when all started non-daemon threads have terminated.

The daemon status of a virtual thread is always `true` and cannot be changed by this method to `false`.

This method must be invoked before the thread is started. The behavior of this method when the thread has terminated is not specified.

Parameters:

`on` - if `true`, marks this thread as a daemon thread

Throws:

`IllegalArgumentException` - if this is a virtual thread and `on` is false

`IllegalThreadStateException` - if this thread is alive

isDaemon

```
public final boolean isDaemon()
```

Tests if this thread is a daemon thread. The daemon status of a virtual thread is always `true`.

Returns:

`true` if this thread is a daemon thread; `false` otherwise.

See Also:

`setDaemon(boolean)`

checkAccess

```
@Deprecated(since="17",
            forRemoval=true)
public final void checkAccess()
```

Deprecated, for removal: This API element is subject to removal in a future version.
This method originally determined if the currently running thread had permission to modify this thread. This method was only useful in conjunction with the Security Manager, which is no longer supported. There is no replacement for the Security Manager or this method.

Does nothing.

toString

```
public String toString()
```

Returns a string representation of this thread. The string representation will usually include the thread's identifier and name. The default implementation for platform threads includes the thread's identifier, name, priority, and the name of the thread group.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this thread.

getContextClassLoader

```
public ClassLoader getContextClassLoader()
```

Returns the context `ClassLoader` for this thread. The context `ClassLoader` may be set by the creator of the thread for use by code running in this thread when loading classes and resources. If not set, the default is to inherit the context class loader from the parent thread.

The context `ClassLoader` of the primordial thread is typically set to the class loader used to load the application.

Returns:

the context `ClassLoader` for this thread, or `null` indicating the system class loader (or, failing that, the bootstrap class loader)

Since:

1.2

setContextClassLoader

```
public void setContextClassLoader(ClassLoader cl)
```

Sets the context `ClassLoader` for this thread.

The context `ClassLoader` may be set by the creator of the thread for use by code running in this thread when loading classes and resources.

Parameters:

`cl` - the context ClassLoader for this Thread, or null indicating the system class loader (or, failing that, the bootstrap class loader)

Since:

1.2

holdsLock

```
public static boolean holdsLock(Object obj)
```

Returns `true` if and only if the current thread holds the monitor lock on the specified object.

This method is designed to allow a program to assert that the current thread already holds a specified lock:

```
    assert Thread.holdsLock(obj);
```

Parameters:

`obj` - the object on which to test lock ownership

Returns:

`true` if the current thread holds the monitor lock on the specified object.

Since:

1.4

getStackTrace

```
public StackTraceElement[] getStackTrace()
```

Returns an array of stack trace elements representing the stack dump of this thread. This method will return a zero-length array if this thread has not

started, has started but has not yet been scheduled to run by the system, or has terminated. If the returned array is of non-zero length then the first element of the array represents the top of the stack, which is the most recent method invocation in the sequence. The last element of the array represents the bottom of the stack, which is the least recent method invocation in the sequence.

Some virtual machines may, under some circumstances, omit one or more stack frames from the stack trace. In the extreme case, a virtual machine that has no stack trace information concerning this thread is permitted to return a zero-length array from this method.

Returns:

an array of `StackTraceElement`, each represents one stack frame.

Since:

1.5

See Also:

`Throwable.getStackTrace()`

getAllStackTraces

```
public static Map<Thread, StackTraceElement[]> getAllStackTraces()
```

Returns a map of stack traces for all live platform threads. The map does not include virtual threads. The map keys are threads and each map value is an array of `StackTraceElement` that represents the stack dump of the corresponding `Thread`. The returned stack traces are in the format specified for the `getStackTrace` method.

The threads may be executing while this method is called. The stack trace of each thread only represents a snapshot and each stack trace may be obtained at different time. A zero-length array will be returned in the map value if the virtual machine has no stack trace information about a thread.

Returns:

a `Map` from `Thread` to an array of `StackTraceElement` that represents the stack trace of the corresponding thread.

Since:

1.5

See Also:

`getStackTrace()`, `Throwable.getStackTrace()`

getId

```
@Deprecated(since="19")
public long getId()
```

Deprecated.
This method is not final and may be overridden to return a value that is not the thread ID. Use `threadId()` instead.

Returns the identifier of this Thread. The thread ID is a positive `long` number generated when this thread was created. The thread ID is unique and remains unchanged during its lifetime.

Returns:

this thread's ID

Since:

1.5

threadId

```
public final long threadId()
```

Returns the identifier of this Thread. The thread ID is a positive `long` number generated when this thread was created. The thread ID is unique and remains unchanged during its lifetime.

Returns:

this thread's ID

Since:

19

getState

```
public Thread.State getState()
```

Returns the state of this thread. This method is designed for use in monitoring of the system state, not for synchronization control.

Returns:

this thread's state.

Since:

1.5

setDefaultUncaughtExceptionHandler

```
public static void setDefaultUncaughtExceptionHandler
(Thread.UncaughtExceptionHandler ueh)
```

Set the default handler invoked when a thread abruptly terminates due to an uncaught exception, and no other handler has been defined for that thread.

Uncaught exception handling is controlled first by the thread, then by the thread's `ThreadGroup` object and finally by the default uncaught exception handler. If the thread does not have an explicit uncaught exception handler set, and the thread's thread group (including parent thread groups) does not specialize its `uncaughtException` method, then the default handler's `uncaughtException` method will be invoked.

By setting the default uncaught exception handler, an application can change the way in which uncaught exceptions are handled (such as logging to a specific device, or file) for those threads that would already accept whatever "default" behavior the system provided.

Note that the default uncaught exception handler should not usually defer to the thread's `ThreadGroup` object, as that could cause infinite recursion.

Parameters:

ueh - the object to use as the default uncaught exception handler. If `null` then there is no default handler.

Since:

1.5

See Also:

`setUncaughtExceptionHandler(Thread.UncaughtExceptionHandler)`, `getUncaughtExceptionHandler()`, `ThreadGroup.uncaughtException(Thread, Throwable)`

getDefaultUncaughtExceptionHandler

```
public
static Thread.UncaughtExceptionHandler getDefaultUncaughtExceptionHandler()
```

Returns the default handler invoked when a thread abruptly terminates due to an uncaught exception. If the returned value is `null`, there is no default.

Returns:

the default uncaught exception handler for all threads

Since:

1.5

See Also:

`setDefaultUncaughtExceptionHandler(Thread.UncaughtExceptionHandler)`

getUncaughtExceptionHandler

```
public Thread.UncaughtExceptionHandler getUncaughtExceptionHandler()
```

Returns the handler invoked when this thread abruptly terminates due to an uncaught exception. If this thread has not had an uncaught exception handler explicitly set then this thread's `ThreadGroup` object is returned, unless this thread has terminated, in which case `null` is returned.

Returns:

the uncaught exception handler for this thread

Since:

1.5

setUncaughtExceptionHandler

```
public void setUncaughtExceptionHandler
(Thread.UncaughtExceptionHandler ueh)
```

Set the handler invoked when this thread abruptly terminates due to an uncaught exception.

A thread can take full control of how it responds to uncaught exceptions by having its uncaught exception handler explicitly set. If no such handler is set then the thread's `ThreadGroup` object acts as its handler.

Parameters:

ueh - the object to use as this thread's uncaught exception handler. If `null` then this thread has no explicit handler.

Since:

1.5

See Also:

`setDefaultUncaughtExceptionHandler(Thread.UncaughtExceptionHandler)`, `ThreadGroup.uncaughtException(Thread, Throwable)`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.